

CSE470: Software Engineering

Design Pattern



What is Design Pattern

- Design patterns represent the best practices used by experienced object-oriented software developers.
- Design patterns are solutions to general problems that software developers faced during software development.
- These solutions were obtained by trial and error by numerous software developers over quite a substantial period of time.



What is Gangs of Four (GOF)

In 1994, four authors Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides published a book titled Design Patterns - Elements of Reusable Object-Oriented Software which initiated the concept of Design Pattern in Software development.

These authors are collectively known as Gang of Four (GOF). According to these authors design patterns are primarily based on the following principles of object oriented design.

- Program to an interface not an implementation
- Favor object composition over inheritance



Usage of Design Pattern

Design Patterns have two main usages in software development.

Common platform for developers

Design patterns provide a standard terminology and are specific to particular scenario. For example, a singleton design pattern signifies use of single object so all developers familiar with single design pattern will make use of single object and they can tell each other that program is following a singleton pattern.

Best Practices

Design patterns have been evolved over a long period of time and they provide best solutions to certain problems faced during software development.

Learning these patterns helps inexperienced developers to learn software design in an easy and faster way.

Types of Design Patterns

As per the design pattern reference book Design Patterns - Elements of Reusable Object-Oriented Software, there are 23 design patterns which can be classified in three categories:

- Creational
- Structural
- Behavioral



Creational

- Talks about object creations.
- To make design more readable and less complexity.
- Talks about efficiently object creation.
- Example: Singleton Pattern



Structural

- Talks about how classes and objects can be structured / composed.
- Parent-child relationship like inheritance used for structuring more than one classes.
- Deals with simplicity in identifying relationships.
- Example: Adapter Pattern



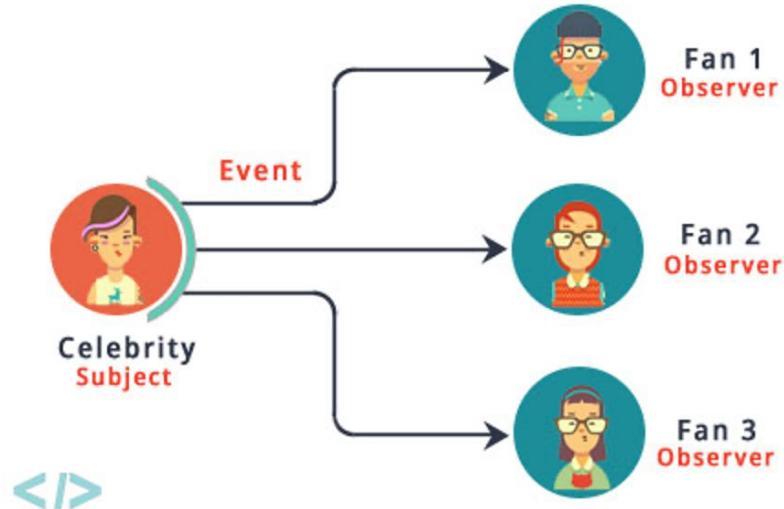
Behavioral

- Deals with interactions and responsibility of objects.
- Used for identifying and setting up common communication patterns among objects.
- Example: Observer Pattern



Observer Pattern

Intent: Define a one-to-many dependency between objects so that when one object change state, all its dependents are notified and updated automatically.

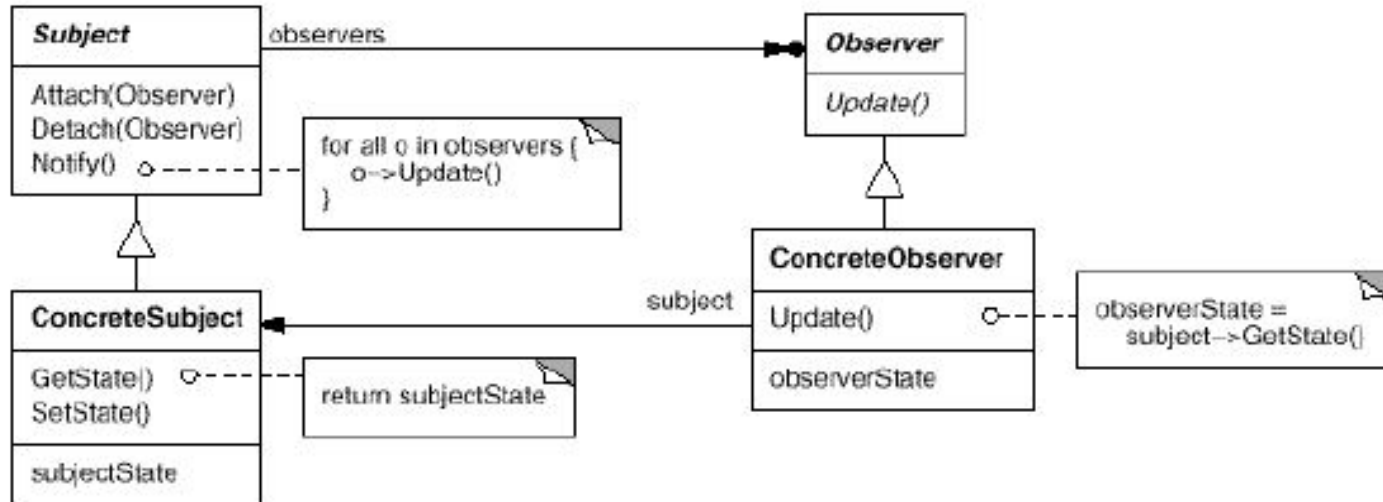


Participants

- **Subject:** knows its observers. Any number of Observer objects may observe a subject. It sends a notification to its observers when its state changes. (ex: Celebrity)
- **Observer:** defines an updating interface for objects that should be notified changes in a subject. (ex: Fans)
- **Concrete Subject:** (ex: FilmCelebrity, FashionCelebrity)
- **Concrete Observer** (ex: FilmFan, FashionFan)



Structure



Observer Pattern

Class Teacher:

```
def __init__(self, name):  
    self.students=[]  
    self.name=name  
def attach_student(self, student):  
    self.students.append(student)  
    print(f"{student.name} has been attached to {self.name}")
```

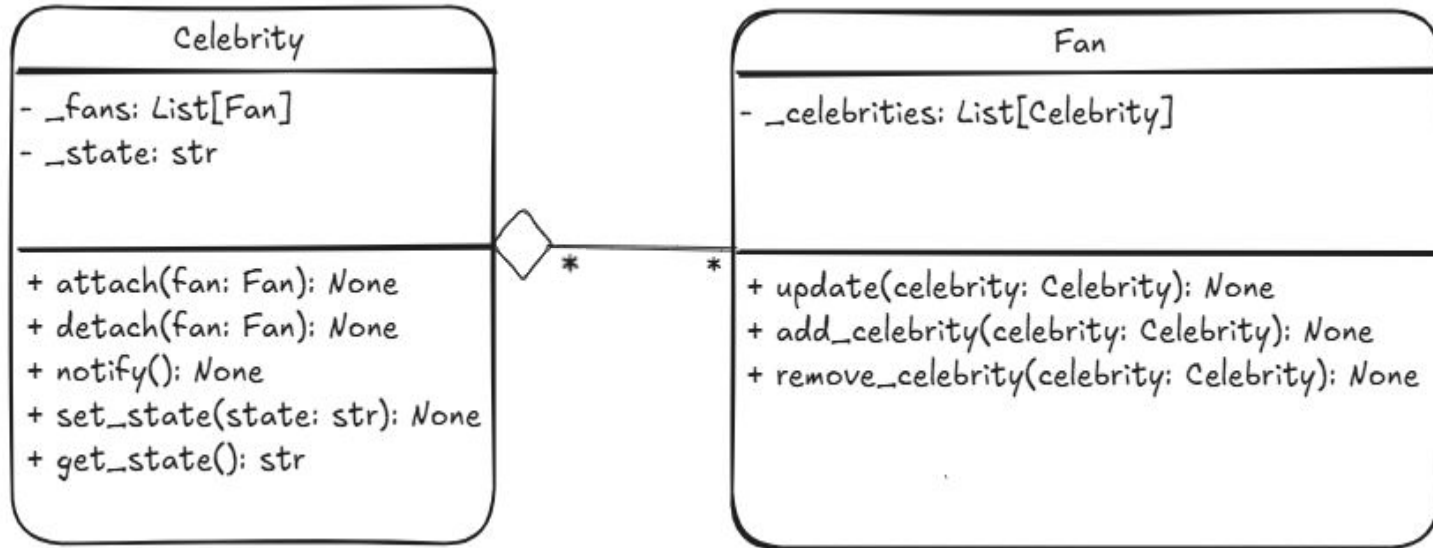
Class Student:

```
def __init__(self,name):  
    self.teachers=[]  
    self.name=name  
def add_teachers(self,teacher):  
    self.teachers.append(teacher)  
    teacher.attach_student(self)
```

driver code

```
teacher_fzn = Teacher("FZN")  
student_ryan = Student("Ryan")  
student_ryan.add_teacher(teacher_fzn)
```

Observer Pattern



Fan-Celebrity Example

Subject (Celebrity)

class Celebrity:

```
def __init__(self):
    self._fans = []
    self._state = None
def attach(self, fan):
    self._fans.append(fan)
def detach(self, fan):
    if fan in self._fans:
        self._fans.remove(fan)
def notify(self):
    for fan in self._fans:
        fan.update(self)
def set_state(self, state):
    self._state = state
    self.notify()
def get_state(self):
    return self._state
```

Observer (Fan)

class Fan:

```
def __init__(self):
    self._celebrities = []
def update(self, celebrity):
    state = celebrity.get_state()
def add_celebrity(self, celebrity):
    self._celebrities.append(celebrity)
    celebrity.attach(self)
def remove_celebrity(self, celebrity):
    self._celebrities.remove(celebrity)
    celebrity.detach(self)
```

#Driver Code

```
celebrity = Celebrity()
fan = Fan()

#fan starts following...
fan.add_celebrity(celebrity)

#celeb changes status and
#notification is received by fan
celebrity.set_state('New state')

#fan can stop following...
fan.remove_celebrity(celebrity)
```

Singleton Pattern

Intent: Ensure a class only has one instance, and provide a global point of access to it.

Motivation: The reason behind is

- More than one instance will result in incorrect program behaviour.
- More than one instance will result the overuse of resources.
- Some classes should have only one instance throughout the system

Classification: Classified as one of the most known Creational Pattern



Singleton Pattern

Any real life example ?



Singleton Pattern

Example :

- Database Connection: You should not open multiple DB connections unnecessarily
- Logger System: One central logging system for entire app
- Configuration Manager: You don't want multiple configs floating around.



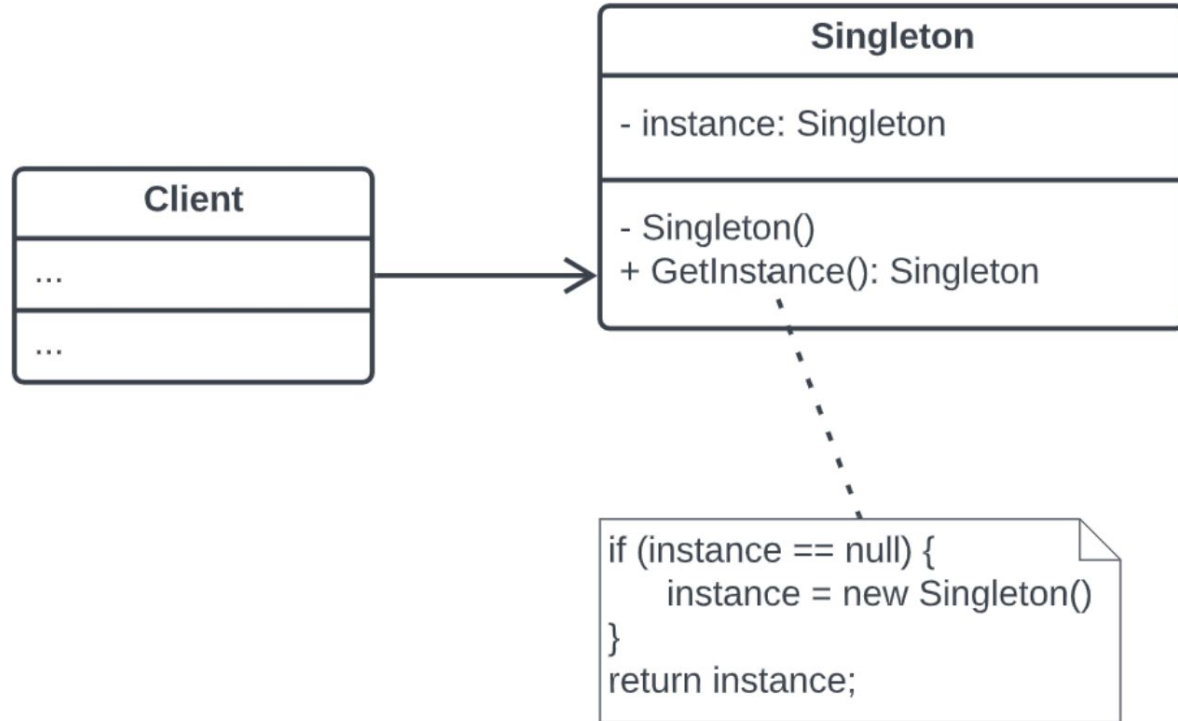
Singleton Pattern

Participants:

Singleton-defines an Instance operation that lets clients access its unique instance. Instance is a class operation .It may be responsible for creating its own unique instance. (ex: Singleton)



Singleton Structure



Singleton Pattern Example

class Singleton:

```
_instance = None
```

```
def __new__(cls):
```

```
    if cls._instance is None:
```

```
        print(' instance creating')
```

```
        cls._instance = super(Singleton, cls).__new__(cls) # return
```

object reference or memory location

```
    return cls._instance
```

class Regular:

```
    def __init__(self):
```

```
        print("new object created")
```

Driver Code

Let's assume that multiple clients
need the service

```
client1 = Singleton()
```

```
client2 = Singleton()
```

```
client3 = Singleton()
```

```
print(client1 is client2 is client3)
```

Now imagine what will happen if we run the driver code for both
Singleton and Regular classes

Why it works

Instantiation: `__new__()` is responsible for creating a new instance of the class. It allocates memory and prepares the object.

Initialization: After `__new__()` returns the new instance, the `__init__()` method is called to initialize the object's state.

class MyClass:

```
def __new__(cls, *args, **kwargs):  
    instance = super(MyClass, cls).__new__(cls)  
    print("Instance created")  
    return instance
```

```
def __init__(self, value):  
    self.value = value  
    print("Instance initialized")
```

#Driver Code

```
obj = MyClass(10)
```

#Output:

```
Instance created  
Instance initialized
```

Lazy Instance

Singleton make use of lazy initiation of the class. It means that its creation is deferred until it is first used. It helps to improve performance, avoid wasteful computation and reduce program memory requirement.

```
if(singleInstance == null){  
    singleInstance = new Singleton(); // Lazy initialization  
}
```

Read more [HERE](#)



Lazy VS Eager Instance

```
# animal.py
# Lazy
class Animal():
    _instance = None

    def __new__(cls):
        if (cls._instance == None):
            cls._instance = super(Animal, cls).__new__(cls)
        return cls._instance

# --- Driver ---
# main.py
from animal import Animal

a = Animal()
a1 = Animal()
```

```
# animal.py
# Eager
class Animal():
    _instance = None

    def __new__(cls):
        if (cls._instance == None):
            cls._instance = super(Animal, cls).__new__(cls)
        return cls._instance

animal_instance = Animal() # Eager initialization
## execute animal class before main function

# --- Driver ---
#main.py
from animal import animal_instance

a = animal_instance
a1 = animal_instance
```

Adapter Pattern

In the real world...

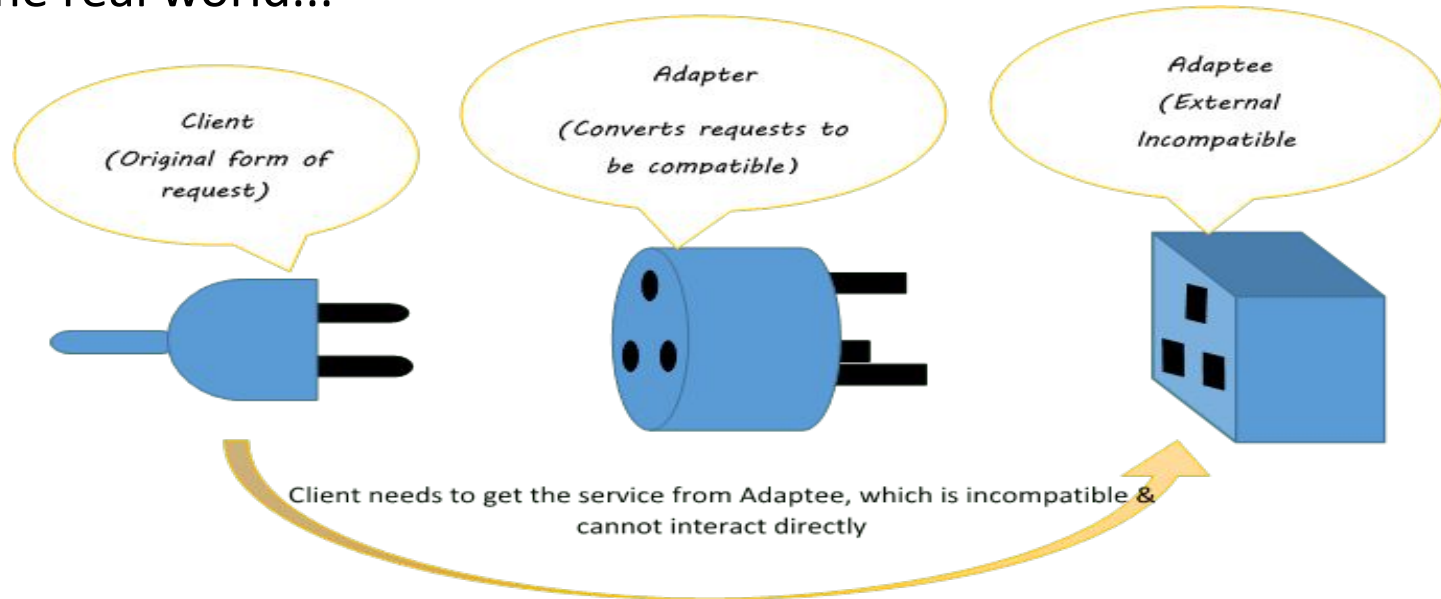
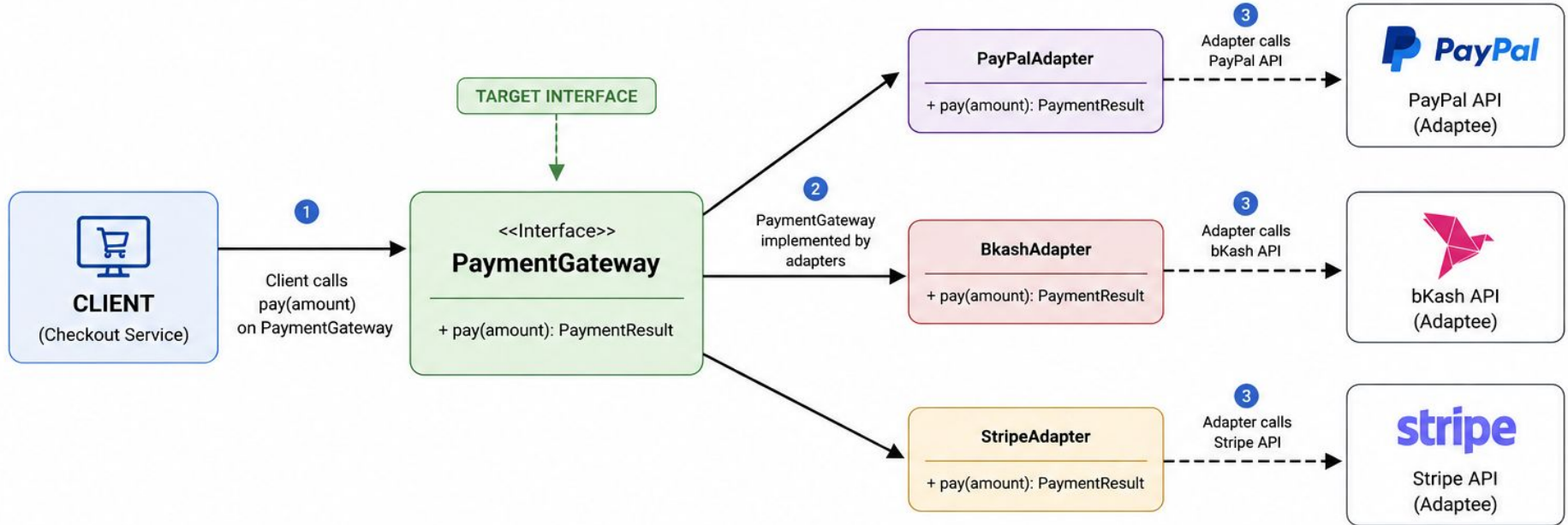


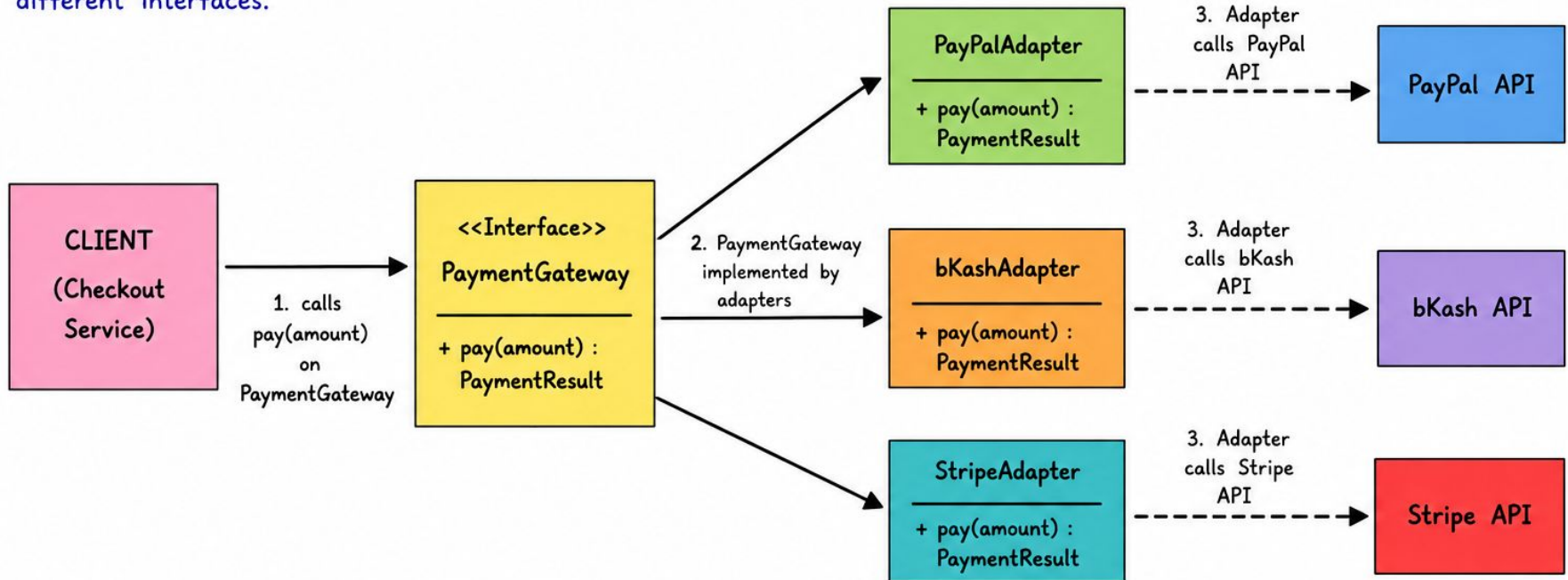
Figure 1-Adapter Pattern Concept

Adapter Pattern



Adapter Pattern

Adapter Pattern allows the client to use a common interface (PaymentGateway) to work with different payment providers (PayPal, bKash, Stripe) that have different interfaces.



Adapter Pattern

Participants:

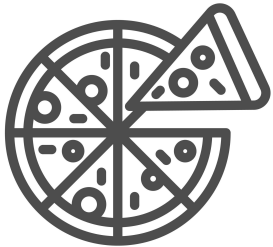
- **Target:** defines the domain-specific interface that Client uses. (ex: Pizza)
- **Client:** collaborates with objects conforming to the Target interface.
- **Adaptee:** defines an existing interface that needs adapting (ex: ChittagongPizza)
- **Adapter:** adapts the interface of Adaptee to the Target interface. (ex: ChittagongClassAdapter)

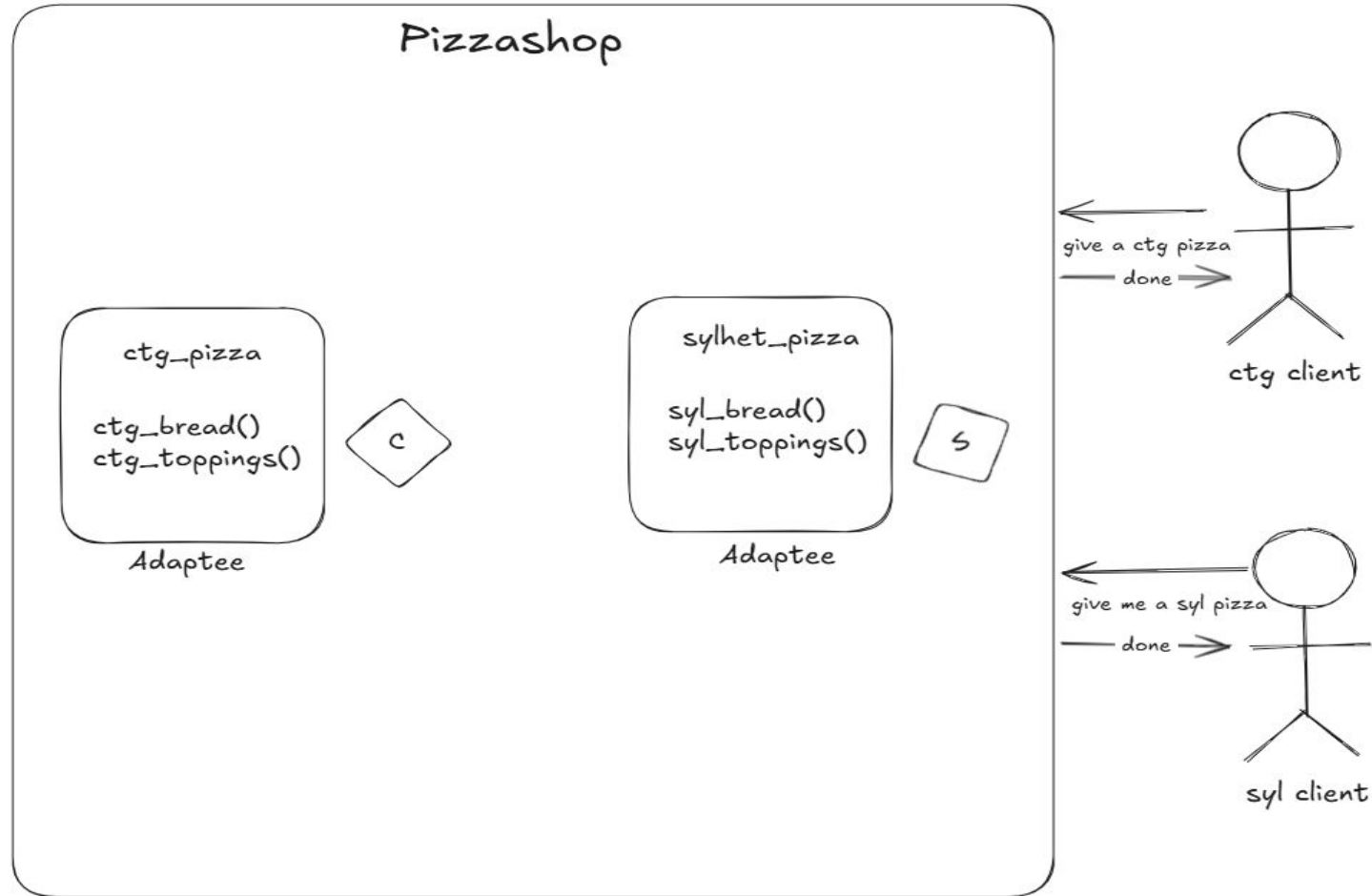


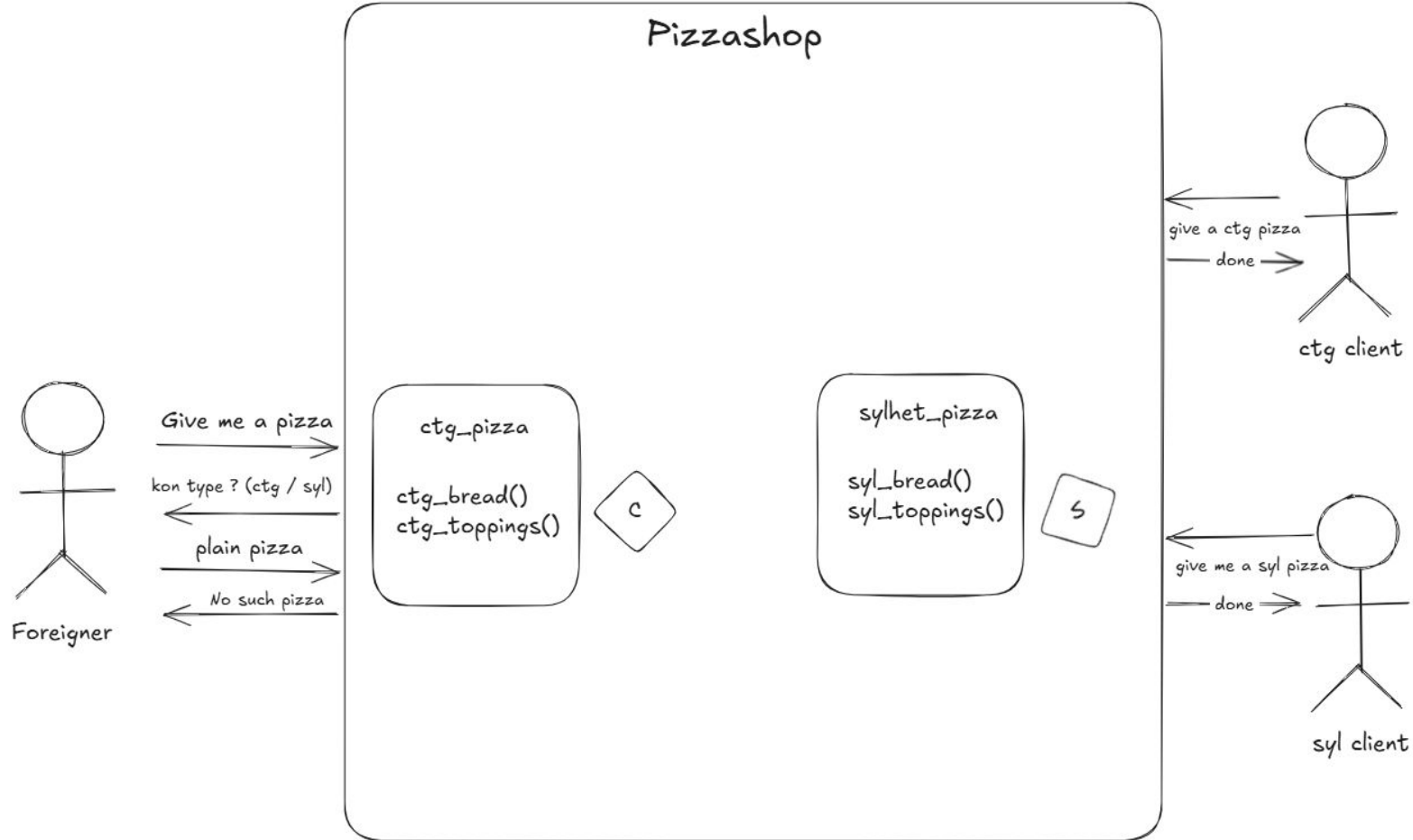
Scenario

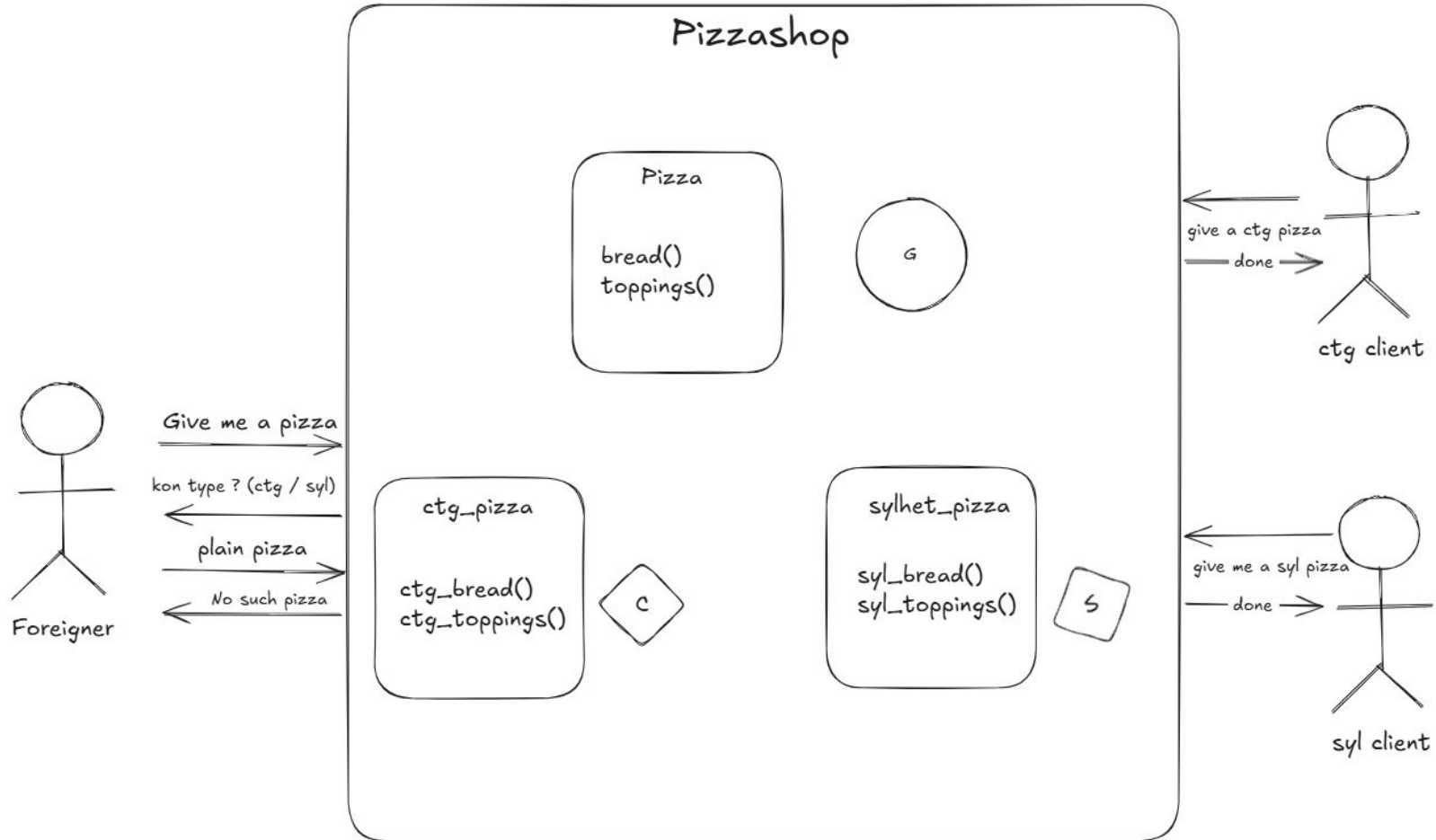
Scenario: I have a pizza making store that creates different pizzas based on the choices of people of different locations. For example – people of Dhaka like DhakaStylePizza, people of Sylhet like SylhetStylePizza.

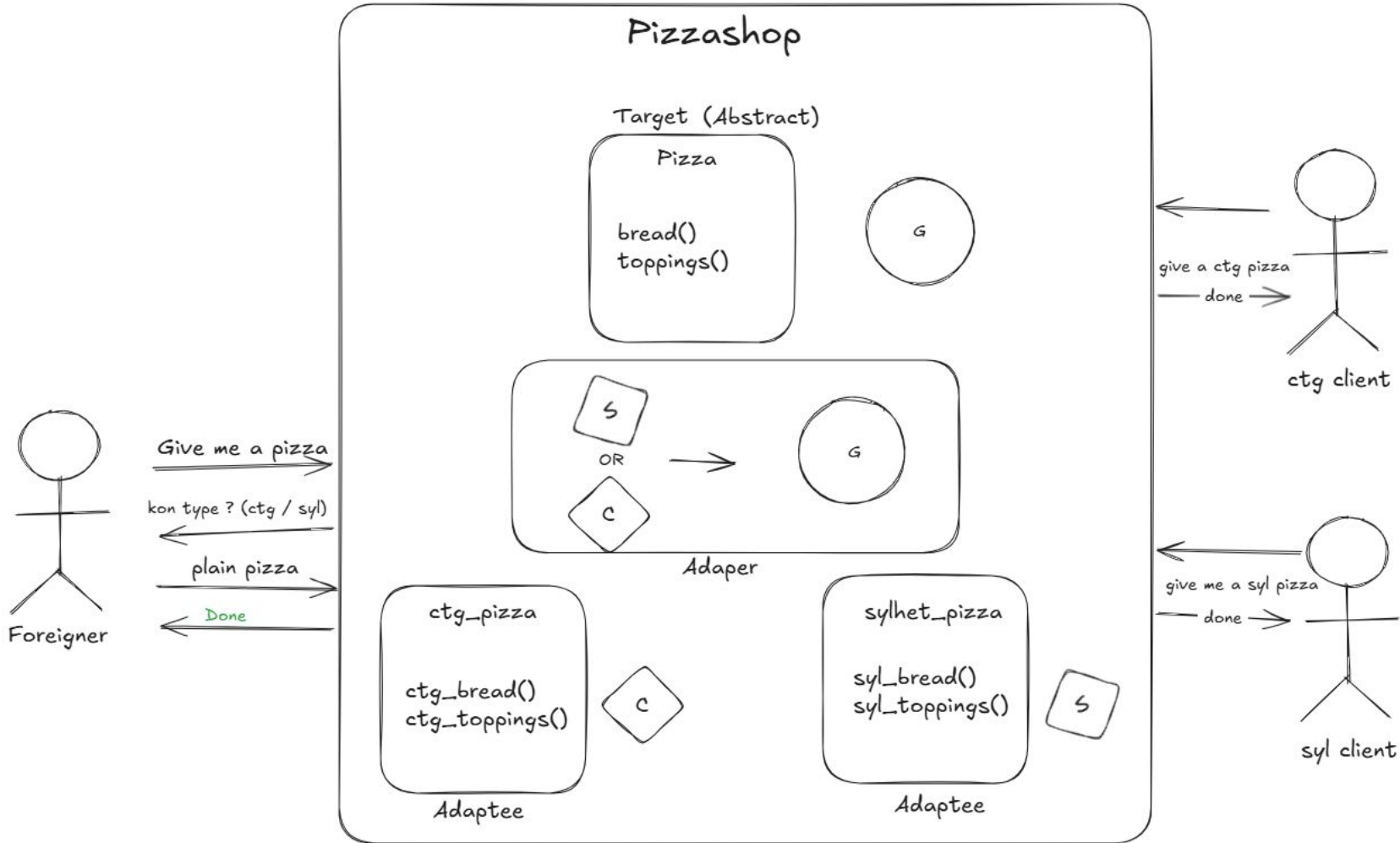
Solution: To meet the scenario, we can declare a Pizza interface and different location people can make their own style pizza by implementing the same interface.











```
from abc import ABC, abstractmethod
```

```
# Target Interface (Pizza interface)
```

```
class Pizza(ABC):
```

```
    @abstractmethod
```

```
    def toppings(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def buns(self):
```

```
        Pass
```

```
# Adaptee: ChittagongPizza
```

```
class ChittagongPizza:
```

```
    def ctg_toppings(self):
```

```
        print("Adding Chatgaiya special toppings")
```

```
    def ctg_buns(self):
```

```
        print("Preparing Chatgaiya style buns")
```

```
# Adaptee: DhakaiyaPizza
```

```
class DhakaiyaPizza:
```

```
    def dhakaiya_toppings(self):
```

```
        print("Adding Dhakaiya special toppings")
```

```
    def dhakaiya_buns(self):
```

```
        print("Preparing Dhakaiya style buns")
```

```
from Pizza import Pizza
```

```
from chittaagongPizzza import ChittagongPizza
```

```
from DhakaPizza import DhakaiyaPizza
```

```
class PizzaAdapter(Pizza):
```

```
    def __init__(self, adaptee):
```

```
        self.adaptee = adaptee
```

```
    def toppings(self):
```

```
        if hasattr(self.adaptee, "ctg_toppings"):
```

```
            self.adaptee.ctg_toppings()
```

```
        elif hasattr(self.adaptee, "dhakaiya_toppings"):
```

```
            self.adaptee.dhakaiya_toppings()
```

```
    def buns(self):
```

```
        if hasattr(self.adaptee, "ctg_buns"):
```

```
            self.adaptee.ctg_buns()
```

```
        elif hasattr(self.adaptee, "dhakaiya_buns"):
```

```
            self.adaptee.dhakaiya_buns()
```

```
# --- Driver Code ---
```

```
if __name__ == "__main__":
```

```
    ctg = PizzaAdapter(ChittagongPizza())
```

```
    dhaka = PizzaAdapter(DhakaiyaPizza())
```

```
    print("Using Chittagong Pizza via Single Adapter:")
```

```
    ctg.toppings()
```

```
    ctg.buns()
```

```
    print("\nUsing Dhakaiya Pizza via Single Adapter:")
```

```
    dhaka.toppings()
```

```
    dhaka.buns()
```

Adapter Pattern Example

```
from abc import ABC, abstractmethod
# Target Interface (Pizza interface)
class Pizza(ABC):
    @abstractmethod
    def toppings(self):
        pass

    @abstractmethod
    def buns(self):
        pass
```

```
# Adaptee: ChittagongPizza
class ChittagongPizza:
    def ctg_toppings(self):
        print("Adding Chatgaiya special toppings")

    def ctg_buns(self):
        print("Preparing Chatgaiya style buns")
```

```
# Adaptee: DhakaiyaPizza
class DhakaiyaPizza:
    def dhakaiya_toppings(self):
        print("Adding Dhakaiya special toppings")

    def dhakaiya_buns(self):
        print("Preparing Dhakaiya style buns")
```



Adapter Pattern Example

```
from abc import ABC, abstractmethod
# Target Interface (Pizza interface)
class Pizza(ABC):
    @abstractmethod
    def toppings(self):
        pass

    @abstractmethod
    def buns(self):
        pass

# Adapter:
class PizzaClassAdapter(Pizza, ChittagongPizza):
    def toppings(self):
        self.ctg_toppings()

    def buns(self):
        self.ctg_buns()
```

```
# Adapter:
class PizzaClassAdapter(Pizza,
DhakaiyaPizza):
    def toppings(self):
        self.dhakaiya_toppings()

    def buns(self):
        self.dhakaiya_buns()
```

Adapter Pattern

Adapter pattern can be solved in one of two ways:

- **Class Adapter:** uses [inheritance](#) and can only wrap a class. It cannot wrap an interface since by definition it must derive from some base class
- **Object Adapter:** uses [composition](#) and can wrap classes or interfaces, or both. It can do this since it contains, as a private, encapsulated member, the class or interface object instance it wraps.



Class Adapter Pattern

Target Interface (Pizza interface)

class Pizza(ABC):

```
@abstractmethod
def toppings(self):
    pass
```

```
@abstractmethod
def buns(self):
    pass
```

class DhakaClassAdapter(Pizza, DhakaiyaPizza):

```
def toppings(self):
    self.dhakaiya_toppings()
```

```
def buns(self):
    self.dhakaiya_buns()
```

We want to adapt the existing DhakaiyaPizza, so it's a Adaptee.
To do so, introduce a Class Adapter, DhakaClassAdapter
Customer use the adapter to adapt to the adaptee.

Object Adapter Pattern

Target Interface (Pizza interface)

class Pizza(ABC):

 @abstractmethod

 def toppings(self):

 pass

 @abstractmethod

 def buns(self):

 pass

Adaptee: DhakaiyaPizza

class DhakaiyaPizza:

 def dhakaiya_toppings(self):

 print("Adding Dhakaiya special toppings")

 def dhakaiya_buns(self):

 print("Preparing Dhakaiya style buns")

#Adapter

class DhakaObjectAdapter(Pizza):

 def __init__(self, adaptee): # ---, dhk
 self.adaptee = adaptee

 def toppings(self):

 self.adaptee.dhakaiya_toppings() #dhk.----

 def buns(self):

 self.adaptee.dhakaiya_buns()

#driver code

dhakaiya_adaptee = **DhakaiyaPizza()**

adapter_dhk=**DhakaObjectAdapter**(dhakaiya_adaptee)

adapter_dhk.toppings()

adapter_dhk.buns()

Useful Links

- [Python Coding Examples](#)
- [Singleton Pattern](#)
- [Observer Pattern](#)
- [Adapter Pattern - 1](#)
- [Adapter Pattern - 2](#)
- [Adapter Pattern - 3](#)

